

그래프의 최소비용 신장트리의 구성

프로젝트 10.1

컴퓨터공학과
20220809 이성재

목차

문제 개요

자료구조 및 알고리즘 설계

알고리즘 분석(시간/공간)

프로그램 구현

실험 결과

논의

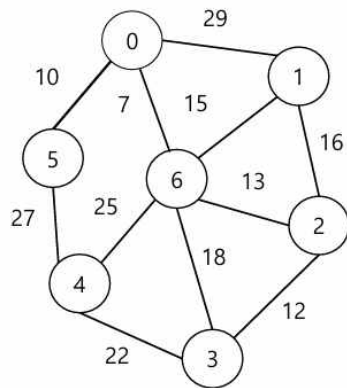
문제 개요

그래프 이론에서 최소비용신장트리(MST: Minimum Spanning Tree)는 연결된 가중치 그래프에서 모든 정점을 포함하면서 간선들의 가중치 합이 최소가 되도록 구성된 트리를 의미한다. 그래프는 일반적으로 사이클을 포함할 수 있는 반면, 트리는 사이클이 존재하지 않는 연결 그래프이며, 모든 정점을 포함하는 사이클 없는 부분 그래프를 신장 트리라 한다. 이러한 신장 트리 중에서도 간선들의 가중치 합이 가장 적은 것을 최소비용신장트리라고 한다.

최소비용신장트리는 컴퓨터 네트워크 구축, 전력선 연결, 도로망 설계 등 다양한 부분에서 최소 비용으로 모든 지점을 연결해야 하는 문제에 활용된다. 따라서 가중치가 주어진 연결 그래프에 최소비용신장트리를 구성하는 문제는 실질적인 최적화 과제로 간주되며, 이를 해결하기 위한 알고리즘으로는 Kruskal 방법과 Prim 방법이 대표적으로 사용된다.

자료구조 및 알고리즘 설계

그래프를 표현하는 방법으로는 인접행렬을 채택했다. 그래프는 정점 간의 연결 정보를 2차원 배열로 표현하는 인접행렬 방식으로 구현하였다. 인접행렬은 `graph[i][j]`에 정점 `i`와 `j`사이의 간선 가중치를 저장하며, 바로 연결되지 않을 경우 ∞ 로 표시한다. 아래는 가중치 그래프를 인접행렬로 표현한 예시이다.



	(0)	(1)	(2)	(3)	(4)	(5)	(6)
(0)	0	29	∞	∞	∞	10	7
(1)	29	0	16	∞	∞	∞	15
(2)	∞	16	0	12	∞	∞	13
(3)	∞	∞	12	0	22	∞	18
(4)	∞	∞	∞	22	0	27	25
(5)	10	∞	∞	∞	27	0	∞
(6)	7	15	13	18	25	∞	0

최소비용신장트리를 구성하는 대표적인 알고리즘은 Prim 방법과 Kruskal 방법이 있다. 이 두 방법 모두 가중치가 있는 연결 그래프에서 모든 정점을 포함하고, 간선의 가중치 합이 최소가 되는 트리 구조를 만든다.

Prim 알고리즘은 정점 중심으로 작동하며, 하나의 정점에서 시작하여 인접한 정점 중 가장 낮은 비용의 간선을 선택하면서 신장 트리를 확장해나가는 그리디(Greedy) 알고리즘이다. 설계 방법은 이러하다.

시작 정점에서 출발하여, 현재까지 선택된 정점 집합과 인접한 간선 중 가장 비용이 낮은 간선을 선택한다. 해당 간선이 연결하는 정점이 아직 트리엔 포함되지 않은 경우, 그 정점을 트리에 포함시킨다. 모든 정점이 포함될 때까지 위 과정을 반복하면 된다.

Kruskal 알고리즘은 전체 간선을 가중치 오름차순으로 정렬한 뒤, 사이클이 생기지 않도록 간선을 하나씩 선택하는 간선 중심의 그리디 방식이다. 과정은 모든 간선을 가중치 기준으로 오름차순 정렬을 하고 정렬된 간선을 하나씩 선택하면서 두 정점이 서로 다른 집합에 속해 있으면 그 간선을 최소비용신장트리에 추가한다. 사이클이 생기지 않도록 Union-Find(Disjoint set) 자료구조를 사용하며 (n-1)개의 간선이 선택되면 종료한다.

최소비용신장트리의 조건은 그래프에 존재하는 모든 정점을 포함해야 하며 신장트리에 포함되는 간선들의 가중치 합이 최소여야 한다. 그리고 신장 트리 내에서 모든 두 정점 간에 경로가 존재해야 하고 동시에 사이클은 존재하지 않아야 한다.

최소비용신장트리를 구하기 위해 Prim 알고리즘을 사용하였다. 아래는 예시 그래프를 Prim 방법으로 정점들을 선택하고 가중치의 총 비용을 계산한 것이다.

Prim 알고리즘 수행 순서 (시작 정점: 0)

1. 정점 0에서 가장 작은 값 7 → 정점 6 선택
2. {0, 6}에서 가장 작은 값 10 → 정점 5 선택
3. {0, 6, 5}에서 가장 작은 값 13 → 정점 2 선택
4. {0, 6, 5, 2}에서 가장 작은 값 12 → 정점 3 선택
5. {0, 6, 5, 2, 3}에서 가장 작은 값 15 → 정점 1 선택
6. {0, 6, 5, 2, 3, 1}에서 가장 작은 값 22 → 정점 4 선택

총 비용: $7 + 10 + 13 + 12 + 15 + 22 = 79$

위의 가중치 그래프를 Prim 알고리즘을 이용하여 최소비용신장트리(MST)를 구성한 결과이며, 그 총 비용은 79이다.

알고리즘 분석(시간/공간)

인접행렬 기반의 Prim 알고리즘을 사용하여 최소비용신장트리(MST)를 구성하였다.

시간 복잡도 분석

정점의 수를 V , 간선의 수를 E 라고 했을 때 각 단계에서 $key[]$ 배열에서 최소값을 찾는 작업이 필요하며, 이는 $O(V)$ 의 시간이

소요된다. 이 과정을 V 번 반복하므로 $O(V) \times O(V)$, 즉 전체 시간복잡도는 $O(V^2)$ 이며, 이는 간선이 많은 밀집 그래프(Dense Graph)에서 적합한 방식이고 구현이 간단하다는 장점이 있다.

공간 복잡도 분석

인접행렬 저장 공간은 $O(V^2)$ 로 정점 간 연결 관계를 2차원 배열로 저장한다. MST 구성용 배열은 $key[V]$, $visited[V]$, $parent[V]$ 각각 $O(V)$ 로 $O(V^2 + V)$ 따라서 전체 공간복잡도는 $O(V^2)$ 이다.

프로그램 구현

```
#include <stdio.h>
#define INF 9999 // 무한대 값 정의

// Prim 알고리즘 함수
void MST_prim(int weight[][100], int n) {
    int selected[100] = {0}; // MST에 포함된 정점 표시
    int edge_count = 0;
    int total_cost = 0;

    selected[0] = 1; // 시작 정점 (0번 선택)

    printf("선택된 간선들 (정점1 - 정점2 : 가중치):\n");

    // n-1개의 간선을 선택할 때까지 반복
    while (edge_count < n - 1) {
        int min = INF;
        int u = -1, v = -1;

        // 현재 MST에 포함된 정점들 중 가장 적은 비용의 간선 선택
        for (int i = 0; i < n; i++) {
            if (selected[i]) {
                for (int j = 0; j < n; j++) {
```

```

        if (!selected[j] && weight[i][j] && weight[i][j] < min) {
            min = weight[i][j];
            u = i;
            v = j;
        }
    }
}

// 선택된 간선 출력 및 MST에 정점 추가
if (u != -1 && v != -1) {
    printf("%d - %d : %d\n", u, v, weight[u][v]);
    selected[v] = 1;
    total_cost += weight[u][v];
    edge_count++;
}
}

// 최소 비용 출력
printf("최소 신장 트리의 총 비용: %d\n", total_cost);
}

int main() {
    int n = 7; // 정점 수
    int weight[100][100] = {
        {0, 29, INF, INF, INF, 10, 7},
        {29, 0, 16, INF, INF, INF, 15},
        {INF, 16, 0, 12, INF, INF, 13},
        {INF, INF, 12, 0, 22, INF, 18},
        {INF, INF, INF, 22, 0, 27, 25},
        {10, INF, INF, INF, 27, 0, INF},
        {7, 15, 13, 18, 25, INF, 0}
    };

    MST_prim(weight, n); // Prim 알고리즘 호출
    return 0;
}

```

실험 결과

실험 목적

Prim 알고리즘을 이용하여 인접행렬로 표현한 가중치 그래프의 최소비용신장트리를 구성하고, 수행 결과를 출력하여 알고리즘의 신장트리의 총 비용과 수행시간을 분석한다.

실행 결과

```
선택된 간선들 (정점1 - 정점2 : 가중치):  
0 - 6 : 7  
0 - 5 : 10  
6 - 2 : 13  
2 - 3 : 12  
6 - 1 : 15  
3 - 4 : 22  
최소 신장 트리의 총 비용 : 79
```

분석 및 해석

정점 수가 7개이므로, 최소비용신장트리는 정확히 6개의 간선으로 구성되어야 한다. 실행 결과, 선택된 간선의 수가 6개로 조건을 만족하며, 총 비용은 79로 계산된 값과 일치한다. 선택된 간선은 사이클을 만들지 않으며, 모든 정점이 연결되어 있어 MST의 조건을 모두 만족한다.

본 구현은 인접행렬과 배열 기반의 Prim 알고리즘으로, 이론적 시간복잡도는 $O(n^2)$ 이다. 하지만 중첩 반복문 구조상 비효율이 쌓이면 실제 수행 시간은 단순히 설명했을 때 $O(n^3)$ 에 가까워질 수 있다. 이 실험에서는 $n=7$ 이므로 총 반복 횟수는 작고 수행 시간이 빠르다. 하지만 정점 수가 많아질 경우 정점수의 제곱해서 커지기 때문에 많은 시간이 걸릴 수 있다. 이럴 경우엔 인접리스트와 우선순위 큐를 사용하는 방법이 더 효율적일 것이다.

논의

가중치 그래프가 주어질 경우 최소비용 신장트리(MST)를 구성하기 위해 Prim 방법을 사용하였다. Prim 알고리즘은 한 정점에서 출발하여 인접한 최소 가중치 간선을 하나씩 선택하면서 신장트리를 확장해 나가는 방법이다. 인접행렬을 이용한 구현이 간단하고, 정점 수가 적을 때 효율적이다.

탐색 과제1: Kruskal 방법을 사용하여 최소비용 신장트리를 구성하였다.

Kruskal 알고리즘은 모든 간선을 가중치 기준으로 오름차순 정렬한 뒤, 사이클이 발생하지 않는 범위 내에서 최소 가중치 간선을 차례로 선택해 MST를 완성하는 방법이다. 주로 간선 리스트를 정렬하고, 서로소 집합(Union-Find) 자료구조를 이용해 사이클 여부를 검사한다.

아래 프로그램은 간선 리스트로 Kruskal 방법을 사용하여 최소비용신장트리를 구성한 것이다.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100
#define INF 9999

typedef struct {
    int u, v;    // 간선의 양 끝 정점
    int weight;  // 간선 가중치
} Edge;

int parent[MAX];

// 서로소 집합 find 함수
int find(int x) {
```

```

    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}

// 서로소 집합 union 함수
void union_set(int a, int b) {
    a = find(a);
    b = find(b);
    if (a != b) parent[b] = a;
}

// 간선 비교 함수 (qsort용)
int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->weight - e2->weight;
}

void MST_kruskal(Edge edges[], int n, int edgeCount) {
    int i, count = 0, total_cost = 0;

    // 초기 서로소 집합 설정
    for (i = 0; i < n; i++) parent[i] = i;

    // 간선 가중치 기준 오름차순 정렬
    qsort(edges, edgeCount, sizeof(Edge), cmp);

    printf("선택된 간선들 (정점1 - 정점2 : 가중치):\n");
    for (i = 0; i < edgeCount && count < n - 1; i++) {
        int uRoot = find(edges[i].u);
        int vRoot = find(edges[i].v);

        if (uRoot != vRoot) { // 사이클이 발생하지 않으면 선택
            union_set(uRoot, vRoot);
            printf("%d - %d : %d\n", edges[i].u, edges[i].v, edges[i].weight);
            total_cost += edges[i].weight;
            count++;
        }
    }

    printf("최소 신장 트리의 총 비용: %d\n", total_cost);
}

```

```

}

int main() {
    int n = 7;
    Edge edges[] = {
        {0, 1, 29}, {0, 5, 10}, {0, 6, 7},
        {1, 2, 16}, {1, 6, 15},
        {2, 3, 12}, {2, 6, 13},
        {3, 4, 22}, {3, 6, 18},
        {4, 5, 27}, {4, 6, 25}
    };
    int edgeCount = sizeof(edges) / sizeof(edges[0]);

    MST_kruskal(edges, n, edgeCount);
    return 0;
}

```

Kruskal 방법을 사용하여 최소비용신장트리를 구한 알고리즘 값이다.

```

선택된 간선들 (정점1 - 정점2 : 가중치):
0 - 6 : 7
0 - 5 : 10
2 - 3 : 12
2 - 6 : 13
1 - 6 : 15
3 - 4 : 22
최소 신장 트리의 총 비용 : 79

```

Prim 방법과 Kruskal 방법 모두 최소비용신장트리를 찾기 위한 알고리즘이기 때문에 같은 가중치 그래프란 조건하에 가중치 합이 같은 것을 알 수 있다.

1. 시간 복잡도

Kruskal 알고리즘의 주요 작업은 두 가지이다.

(1) 간선 정렬

그래프에 포함된 모든 간선을 가중치 오름차순으로 정렬해야 한다.
이 정렬에 걸리는 시간은 $O(E \log E)$ 이다. (E 는 간선 수)

(2) 간선을 하나씩 선택하며 사이클이 생기는지 확인

선택한 두 정점이 서로 다른 집합에 속해 있으면 간선을 선택하고,
같은 집합이면 선택하지 않는다. 이 과정을 위해 Union-Find 구조를
사용한다. 각 간선마다 최대 $O(\log V)$ 이하의 시간이 걸리며,
전체적으로는 $O(E \log V)$ 정도로 볼 수 있다. 하지만 일반적으로는 간선
정렬 $O(E \log E)$ 이 가장 큰 영향을 미치기 때문에, 전체 시간 복잡도는
 $O(E \log E)$ 라고 볼 수 있다.

2. 공간 복잡도

그래프의 모든 간선 정보를 저장해야 하므로 $O(E)$ 공간 필요하다.
그리고 유니온 파인드를 위한 배열(부모, 랭크 등)을 저장하는 데 $O(V)$
공간 필요하다. 이 둘을 더해서 공간 복잡도는 $O(V + E)$ 라고 볼 수
있다.

탐색 과제2: 두 방법의 장단점 비교

Prim 알고리즘은 하나의 정점에서 시작해 최소 비용의 간선을
선택하며 트리를 확장하는 방식이다. 이 방식은 그래프가 밀집된
경우(간선이 많은 경우) 유리하며, 인접행렬을 이용하면 구현이
단순하다. 그러나 희소 그래프에서는 불필요한 간선 검사가 많아
비효율적일 수 있고, 정점 수가 많아지면 성능이 저하된다.

Kruskal 알고리즘은 모든 간선을 가중치 순으로 정렬한 뒤, 사이클이
생기지 않도록 선택해 나가는 방식이다. 이 방식은 희소 그래프에서

효과적이며, 정점 수보다 간선 수가 적을 때 빠르게 동작한다. 하지만 간선 정렬 시간이 추가로 필요하고, 사이클을 방지하기 위한 Union-Find 자료구조를 사용해야 하므로 구현이 다소 복잡할 수 있다.

결론적으로, Prim은 밀집 그래프에, Kruskal은 희소 그래프에 적합하며, 그래프의 구조에 따라 적절히 선택해야 한다.